Practica 2: Diseño automático de sistemas fiables.

Jorge Callado, Pablo Rayón, Fernando Perez

UNIVERSIDAD NEBRIJA

1: Generador de reloj síncrono

1.1 fsm_sclk.vhd

Este es el archivo que define el funcionamiento del generador de reloj síncrono, primero, definimos los puertos de entrada y salida:

```
entity fsm_sclk is
    generic(
        g_freq_SCLK_KHZ : integer := 1500000;
        g_system_clock  : integer := 100000000
    );
    port (
        clk         : in std_logic;
        rst_n       : in std_logic;
        start       : in std_logic;
        SCLK        : out std_logic;
        SCLK_rise   : out std_logic;
        SCLK_fall   : out std_logic
    );
end fsm_sclk;
```

De entrada la entidad recibe la señal de start, reset y el reloj, de salida, la señal de reloj nueva (SCLK), una señal que emitirá un pulso cuando sube SCLK y otra que lo emitirá cuando baje.

Después, también hay que definir las distintas señales auxiliares

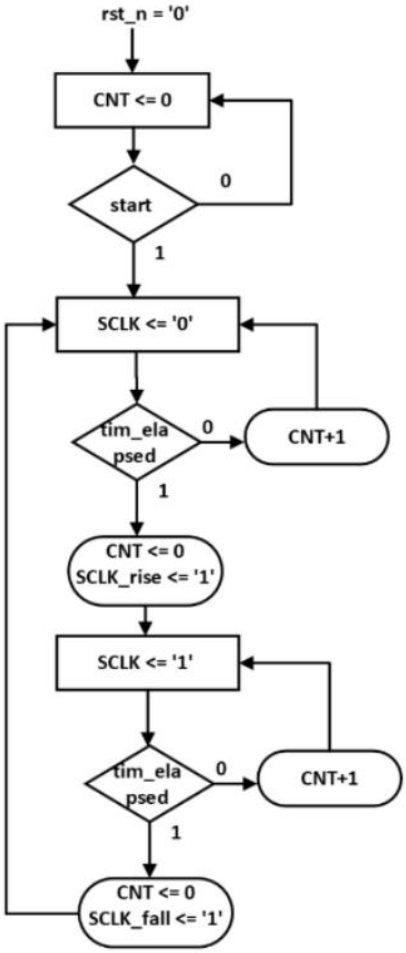
```
architecture behavioural of fsm_sclk is
    constant c_half_T_SCLK : integer := integer (floor(real(g_system_clock) / real( g_freq_SCLK_KHZ )));
    constant c_counter_width : integer := integer(ceil(log2(real(c_half_T_SCLK))));
    signal count : integer:=0;

    type state_type is (IDLE,SCLK0,SCLK1);
    signal current_state,next_state:state_type;

    signal time elapsed, enable count,SCLK f aux,SCLK r aux: std logic;
```

Aquí hay que destacar los estados ya que serán los mas importantes en nuestra arquitectura, el sistema tiene 3 estados, el de reposo y para cuando el reloj este funcionando, sclk0 y sclk1.

Por ultimo, dentro de la arquitectura tenemos 2 procesos, el primero se encarga del funcionamiento del contador y del cambio de estados y el segundo proceso se encarga de la selección del siguiente estado, el funcionamiento de ambos se puede resumir explicando el diagrama asm de esta practica:

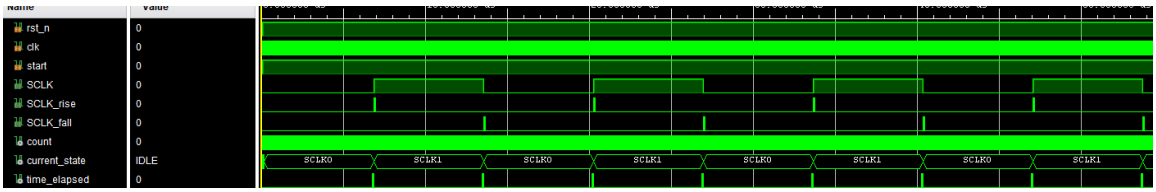


El primer estado corresponde a IDLE, se llega a el cuando se desactiva el rst y la única salida de este es activando la señal start, después se llega a sclk0 que, en cada ciclo de reloj, comprobara si la señal time elapased ya se ha activado y si no es el caso subirá el contador, repetirá este proceso hasta que el contador suba lo suficiente para activar time elapsed, cuando esto ocurra, pasaremos a sclk1, activando la señal sclk y por lol tanto, sclk rise, el ultimo estado tiene el mismo funcionamiento pero al llegar al tiempo objetivo, se desactiva sclk y por lo tanto el pulso lo emite sclk fall

1.2 fsm_tb.vhd

Para el testbench, basta con activar el start, desactivar rst y generar una señal de reloj, como se ve, esto genera una señal de reloj síncrona con su rise y fall

```
process begin
    rst_n_tb <= '0'; wait for 50ns;
    rst_n_tb <= '1'; wait;
end process;
 process begin
        start_tb <= '0'; wait for 60ns;
        start_tb <= '1'; wait;
end process;
process begin
    clk_tb <= '0'; wait for 50ns;
    clk_tb <= '1'; wait for 50ns;
end process;
```



2:Register shifter

2.1 Register_shifter.vhd

Los puertos del dispositivo son:

```
generic(
    N:integer := 4
);
port ( clk   : in std_logic;
       rst_n : in std_logic;
       din   : in std_logic;
       s     : in std_logic_vector (1 downto 0);
       D     : in unsigned(N-1 downto 0);
       dout  : out std_logic;
       Q     : out unsigned(N-1 downto 0)
    );
```

N será el tamaño del registro en el que almacenaremos los datos, después, como puertos de entrada y salida tenemos:

-Reset y clock

-Entrada y salida en serie (din/dout)

-Entrada y salida en paralelo (D/Q)

-S 2 señales lógicas que seleccionaran la acción que realizara el sistema

Las acciones que realizara el sistema según el valor de s serán:

```
if(rising_edge (clk) )then
    case S is
    when "10"   =>
        registro<= din &registro(N-1 downto 1);
    when "11"   =>
        registro <= D;
    when others =>
        registro <= registro;
    end case;
```

s0 nos funciona como enable así que si s0 es 0 no cambiaremos nada, si s0 es 1 y s1 es 0 se moverán todos los valores del registro, eliminando el ultimo dato y metiendo el nuevo desde din, si s=11, metemos todos los datos de la entrada en paralelo al registro

por ultimo, la salida en paralelo sacara todos los valores del registro y dout sacara el ultimo dato del registro.

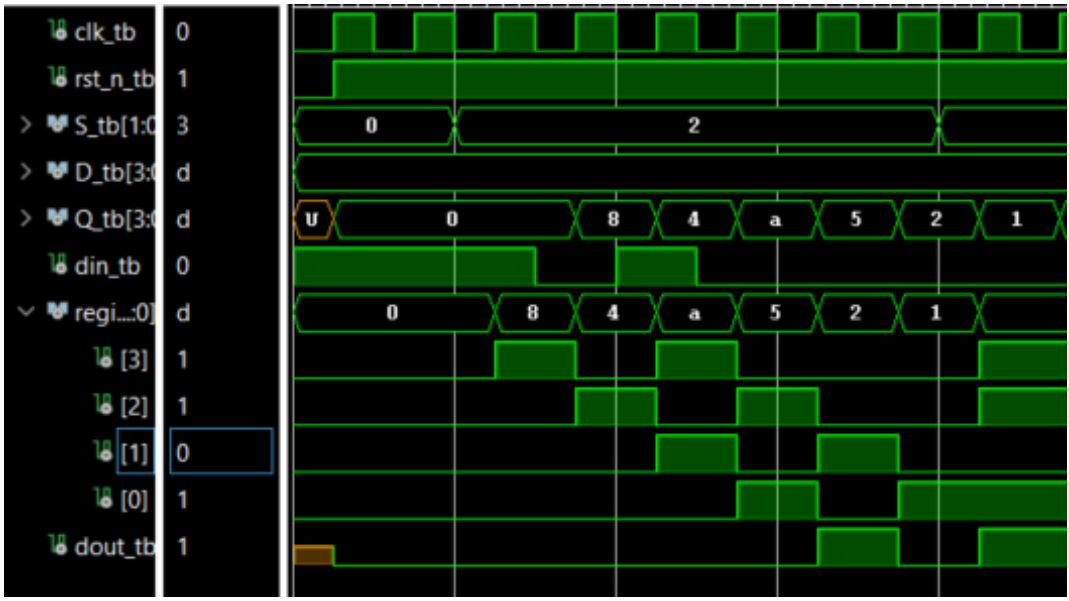
2.2 Register_shifter_tb.vhd

El testbench consiste en una serie de instrucciones:

```
process begin
    rst_n_tb <='0'; wait for 50ns;
    rst_n_tb <= '1' ;wait;
end process;

process begin
    clk_tb <= '0'; wait for 50ns;
    clk_tb <= '1'; wait for 50ns;
end process;

process begin
    D_tb <= "1101";
    din_tb <= '1';
    S_tb <= "00";    wait for 200ns;
    S_tb <= "10";    wait for 100ns;
    din_tb <= '0';   wait for 100 ns;
    din_tb <= '1';   wait for 100ns;
    din_tb <= '0';   wait for 300 ns;
    S_tb <= "11";    wait;
end process;
d Structural;
```



Para empezar, mientras s sea 0, el registro esta vacío aunque haya un dato en din, en el segundo ciclo de reloj s cambia a 2 (10) los datos de din van entrando al registro y este se desplaza para meterlos, después, s cambia a 3 por lo que el registro mete todos los datos de la entrada en paralelo (D) con valor d.

La salida dout siempre saca el ultimo valor del registro y Q el registro entero, ambas con un ciclo de reloj de retraso